Q1. What is an abstract class?

An abstract class is a partially implemented class that serves as a base for other classes. It cannot be instantiated directly because it may contain abstract methods without implementations. An abstract class can have constructors, typically declared as protected or public, which are used to initialize subclass objects.

Q2. What can an abstract class contain?

An abstract class can contain abstract methods, concrete methods, instance variables, and static members. Therefore, it is incorrect to say that an abstract class can only contain abstract methods.

Q3. What are default modifiers in an interface?

Interface methods are public and abstract by default, while interface variables are public, static, and final. Interfaces do not have constructors because they are not designed to be instantiated.

Q4. Multiple inheritance rules in Java

A class cannot extend multiple classes, but it can implement multiple interfaces. This design allows Java to support multiple behaviors without the complexity of multiple class inheritance.

Q5. Interface inheritance

An interface can extend another interface using the extends keyword. It can also extend multiple interfaces, supporting multiple inheritance at the interface level.

Q6. Abstract class vs interface

Abstract classes represent an 'is-a' relationship and can contain state and constructors, while interfaces define capabilities and support multiple inheritance.

Q7. Why Java avoids multiple class inheritance

Java avoids multiple class inheritance to prevent ambiguity such as the Diamond Problem. Interfaces avoid this issue by not maintaining object state.

Q8. Aggregation vs Composition

Composition represents strong ownership where the child object depends on the parent for its lifecycle, while aggregation represents weak ownership where objects can exist independently.

Q9. Favor composition over inheritance

Composition is more flexible than inheritance because it allows behavior changes at runtime and reduces tight coupling between classes.

Q10. Singleton pattern characteristics

The Singleton pattern ensures a single instance, global access, and controlled instantiation. The static inner class approach provides thread safety and lazy initialization through JVM class loading.